String Matching Algorithms: Finite Automata and KMP

Sofiia Kotsiubynska

Contents

1 How Finite Automaton Matching Works **2**
1.1 The Idea 2
1.2 Code analyzis 2

2 The Knuth-Morris-Pratt (KMP) Algorithm **3**
2.1 The Idea 3
2.2 Code analyzis 4

1 How Finite Automaton Matching Works

The Finite Automaton approach turns our search pattern into a simple state machine. We can think of each "state" as a score that tracks your current matching progress. It represents exactly how many letters of the pattern have been successfully matched in a row so far.

1.1 The Idea

If a pattern is m characters long, the machine uses $m + 1$ states to track progress:

- **State** 0 **(Start):** No characters have matched yet. The machine is looking for the very first letter of the pattern.
- **State** k **(Partial Match):** The first k characters have matched perfectly. The machine is now looking for character $k + 1$.
- **State** m **(Success):** Every single letter of the pattern has matched in a row. The machine stops and returns the match location.

If the next letter in the text matches the next expected letter of the pattern, the match streak continues. The machine adds 1 to the state score and moves forward:

$$\text{Next State} = \text{Current State} + 1 \tag{1}$$

If a letter does not match, a basic search algorithm would lose all progress and reset completely back to State 0. The automaton is smarter. It looks at the letters it has already matched, adds the new mismatching letter to the end, and checks if the *start* of the pattern matches the *end* of this text window.

For example, if we are looking for the pattern **"ACACAGA"** and we are at **State 5** (we have already matched "ACACA"), but the next letter is a mismatching **"C"**, the history window becomes **"ACACAC"**. The beginning of our pattern ("ACAC") matches the end of this window perfectly. Instead of resetting to 0, the machine safely falls back to **State 4** and keeps searching.

1.2 Code analyzis

The Prolog implementation manages this state machine using the following components:

- `fa_search/3`**:** The main entry point. It breaks the text and pattern strings into clean lists of characters and starts the search loop.
- `fa_run_search/6`**:** The main loop that steps through the text. If the current state score equals the pattern length, it stops and calculates the matching index.
- `compute_next_state/5`**:** It decides what state to visit next:
  - Rule 1 moves the state up by 1 if there is a match.
  - Rule 2 calculates the fallback state if there is a mid-stream mismatch.
  - Rule 3 keeps the machine at state 0 if it misfires on the very first letter.
- `find_fallback_state/4`**:** When a mismatch happens, this looks backward to find the longest historical overlap so the machine doesn't lose all progress.
- `take/3`**:** A simple utility that clips a specific number of items from the front of a list to help check sub-patterns.

2 The Knuth-Morris-Pratt (KMP) Algorithm

The Knuth-Morris-Pratt (KMP) algorithm shares the exact same goal as the Finite Automaton: it scans text from left to right without ever moving the text pointer backward. However, KMP is much lighter on memory.

While an automaton builds a large transition table for every single letter in the alphabet, KMP only looks at the pattern itself to build a simple, one-dimensional array called the **Longest Prefix Suffix (LPS) Table**.

2.1 The Idea

To see why KMP is useful, imagine searching for the pattern **"AAAA"** inside the text **"AAAAAB"**.

A naive search checks the first four letters:

```
Text:  A A A A A B
Pattern:  A A A A
```

This is a match. A naive search would then slide forward by just one position and start completely over:

```
Text:  A A A A A B
Pattern:  A A A A
```

It re-compares the first three **"A"**s manually, forgetting that it just saw them. KMP eliminates this waste. It remembers the repeating patterns inside the string, recognizes that those three letters are guaranteed to match, and skips directly to checking the 4th letter.

The `LPS` table tracks repeating structures within the pattern. For any position i, `LPS[i]` stores the length of the longest starting part (prefix) of the pattern that is identical to the ending part (suffix) of that substring.

Here is how the table is built for the pattern **"aabaaac"** step-by-step:

- **Index 0 ("a"):** Single letters have no overlap. `LPS[0] = 0`.
- **Index 1 ("aa"):** The prefix **"a"** matches the suffix **"a"**. `LPS[1] = 1`.
- **Index 2 ("aab"):** No prefix matches the trailing **"b"**. `LPS[2] = 0`.
- **Index 3 ("aaba"):** The prefix **"a"** matches the final **"a"**. `LPS[3] = 1`.
- **Index 4 ("aabaa"):** The prefix **"aa"** matches the final **"aa"**. `LPS[4] = 2`.
- **Index 5 ("aabaaa"):** The prefix **"aa"** matches the final **"aa"**. `LPS[5] = 2`.
- **Index 6 ("aabaaac"):** The trailing **"c"** breaks the pattern. `LPS[6] = 0`.

The final calculated table is `[0, 1, 0, 1, 2, 2, 0]`.

To build this table, the algorithm steps through the pattern using two pointers: a tracking pointer (`Prefix_Length`, starting at 0) and an exploring pointer (`Current_Index`, starting at 1). It follows three simple rules:

1. **Match:** If the characters match, the pattern streak is growing. Increment `Prefix_Length` by 1, save it in the table at the current index, and move forward.
2. **Mismatch Fallback:** If they do not match and `Prefix_Length > 0`, look at the previous table value (`LPS[Prefix_Length - 1]`) to see if a shorter repeating pattern fits, then try again.

3. **Dead End:** If they do not match and `Prefix_Length` is already 0, save a `0` in the table and move to the next index.

When searching live text, KMP uses a text pointer (i) and a pattern pointer (j):

- **Match:** If the letters match, advance both pointers by 1. If the pattern pointer reaches the end, a full match is recorded.
- **Mismatch (with progress):** If the letters do not match but $j > 0$, keep the text pointer completely still. Slide the pattern pointer back using the table data: $j =$ `LPS[j - 1]`.
- **Mismatch (at start):** If a mismatch happens on the very first letter of the pattern ($j == 0$), leave the pattern pointer at 0 and move the text pointer forward by 1 to try the next spot.

2.2 Code analyzis

The KMP algorithm organizes these operations across these core functions:

- `kmp_search/3`**:** Initializes the program. It converts strings into list arrays, triggers the LPS preprocessor, and boots the search loop.
- `build_lps_table/2`**:** Prepares the LPS table structure, locks the first value to `0`, and starts the table generation loop.
- `build_lps_loop/4`**:** Runs the preprocessor logic. Case 1 stops when the pattern is fully evaluated; Case 2 records character matches; Case 3 manages partial fallbacks; Case 4 handles dead ends by entering a 0.
- `kmp_run_search/6`**:** It keeps track of the text and pattern pointers, records success indices, handles matches, updates fallback alignments, and moves the text window forward.

References

[1] GeeksforGeeks, *Finite Automata algorithm for Pattern Searching*,
Available at: https://www.geeksforgeeks.org/dsa/finite-automata-algorithm-for-pattern-searc

[2] GeeksforGeeks, *KMP Algorithm*,
Available at: https://www.geeksforgeeks.org/dsa/kmp-algorithm-for-pattern-searching/